

Array Methods

A beginner-friendly reference for the 8 most important array methods

Method	What it does	Returns
map()	Transform every item	New array (same length)
find()	Get first matching item	Single item or undefined
findIndex()	Get index of first match	Number (-1 if not found)
filter()	Keep items that pass a test	New array (shorter/same)
reduce()	Boil array down to one value	Any single value
concat()	Join two or more arrays	New combined array
slice()	Copy a portion (non-destructive)	New array (portion)
splice()	Add / remove items in place	Array of removed items

1 · map()

Transforms • Non-destructive • Returns new array

map() goes through every item in an array, runs your function on it, and collects the results into a brand-new array. The original array is never changed.

Syntax:

```
const newArray = array.map( (item, index, arr) => { ... } );
```

Parameters:

Parameter	Description
item	The current element being processed
index	(Optional) Position of the current element
arr	(Optional) The full original array

Example:

```
const prices = [100, 200, 300];

// Add 18% GST to every price
const withGST = prices.map( price => price * 1.18 );

console.log(withGST);
console.log(prices); // original is unchanged
```

■ [118, 236, 354]

■ [100, 200, 300]

■ Think of map() as a factory conveyor belt — every item goes in, gets transformed, comes out the other side.

2 · find()

Search • Returns first match • Stops early

`find()` searches the array and returns the very first item that passes your test. If nothing matches it returns `undefined`. It stops as soon as it finds a match — so it's fast.

Syntax:

```
const item = array.find( (item, index, arr) => { ... } );
```

Parameters:

Parameter	Description
item	The current element being checked
index	(Optional) Position of the current element
arr	(Optional) The full original array

Example:

```
const users = [  
  { id: 1, name: 'Priya' },  
  { id: 2, name: 'Rohan' },  
  { id: 3, name: 'Anita' },  
];  
  
const user = users.find( u => u.id === 2 );  
console.log(user);  
  
const missing = users.find( u => u.id === 99 );  
console.log(missing);
```

```
■ { id: 2, name: 'Rohan' }
```

```
■ undefined
```

■ Use `find()` when you expect one result (like fetching a user by ID). For multiple results, use `filter()` instead.

3 · findIndex()

Search • Returns index number • -1 if not found

findIndex() works just like find(), but instead of returning the item itself it returns its position (index) in the array. Returns -1 when no match is found.

Syntax:

```
const idx = array.findIndex( (item, index, arr) => { ... } );
```

Parameters:

Parameter	Description
item	The current element being checked
index	(Optional) Position of the current element
arr	(Optional) The full original array

Example:

```
const fruits = ['apple', 'banana', 'mango', 'orange'];

const idx = fruits.findIndex( f => f === 'mango' );
console.log(idx);

const notFound = fruits.findIndex( f => f === 'grape' );
console.log(notFound);
```

■ 2

■ -1

■ Handy when you need to update or delete an item: find its index first, then use splice() to modify it.

4 · filter()

Select • Non-destructive • Returns new array

`filter()` tests every item with your function. Items that return `true` are kept; items that return `false` are dropped. The result is a new array — the original stays the same.

Syntax:

```
const newArray = array.filter( (item, index, arr) => { ... } );
```

Parameters:

Parameter	Description
item	The current element being tested
index	(Optional) Position of the current element
arr	(Optional) The full original array

Example:

```
const scores = [45, 72, 38, 91, 55, 60];

// Keep only passing scores (>= 50)
const passed = scores.filter( s => s >= 50 );
const failed = scores.filter( s => s < 50 );

console.log(passed);
console.log(failed);
```

■ [72, 91, 55, 60]

■ [45, 38]

■ *filter() is like a sieve — pour the whole array in, only what passes the test comes out.*

5 · reduce()

Accumulate • Returns single value • Most powerful

`reduce()` processes each item and builds up a single result — a total, an object, a string, anything. You give it a starting value (accumulator) and a function that combines the accumulator with each item, one by one.

Syntax:

```
const result = array.reduce( (accumulator, item, index, arr) => { ... }, initialValue );
```

Parameters:

Parameter	Description
accumulator	The running result carried from the previous call
item	The current element being processed
initialValue	Starting value for the accumulator (recommended always)

Example:

```
const cart = [
  { name: 'Shirt', price: 799 },
  { name: 'Jeans', price: 1499 },
  { name: 'Shoes', price: 2299 },
];

// Calculate total bill
const total = cart.reduce( (sum, item) => sum + item.price, 0 );
console.log('Total: ■' + total);
```

```
■ Total: ■4597
```

■ Always pass an *initialValue* (the second argument). Without it, `reduce()` uses the first array item — which can cause bugs with empty arrays.

6 · concat()

Combine • Non-destructive • Returns new array

`concat()` merges two or more arrays together and returns a brand-new array. The original arrays are never modified. You can also use the spread operator (...) as a modern alternative.

Syntax:

```
const combined = array1.concat(array2, array3, ...);
```

Parameters:

Parameter	Description
array2, array3...	One or more arrays (or values) to append

Example:

```
const boys = ['Arjun', 'Karan'];
const girls = ['Priya', 'Meera'];
const extra = ['Sam'];

const all = boys.concat(girls, extra);
console.log(all);

// Modern alternative using spread
const all2 = [...boys, ...girls, ...extra];
console.log(all2);
```

```
■ ['Arjun', 'Karan', 'Priya', 'Meera', 'Sam']
■ ['Arjun', 'Karan', 'Priya', 'Meera', 'Sam']
```

■ *`concat()` is perfect for merging API results from multiple pages without mutating your existing data.*

7 · slice()

Extract • Non-destructive • Returns new array

`slice()` cuts out a portion of an array and returns it as a new array. You give it a start index (inclusive) and an end index (exclusive). The original array is untouched. Negative indices count backwards from the end.

Syntax:

```
const portion = array.slice(start, end);
```

Parameters:

Parameter	Description
start	Index to begin from (inclusive). Negative counts from end.
end	(Optional) Index to stop before (exclusive). Omit to go to the end.

Example:

```
const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'];

console.log( months.slice(0, 3) ); // first 3
console.log( months.slice(2) ); // from index 2 to end
console.log( months.slice(-2) ); // last 2
console.log( months.slice(1, 4) ); // index 1 up to (not incl.) 4
console.log( months ); // original unchanged
```

```
■ ['Jan', 'Feb', 'Mar']
■ ['Mar', 'Apr', 'May', 'Jun']
■ ['May', 'Jun']
■ ['Feb', 'Mar', 'Apr']
■ ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
```

■ *`slice()` with no arguments (`array.slice()`) creates a shallow copy of the whole array — useful when you need a duplicate.*

8 · splice()

Modify in-place • Destructive • Add & Remove

splice() changes the original array by removing, replacing, or inserting items. It returns an array of the removed items. Unlike slice(), this method mutates (changes) the original array directly — so use it with care.

Syntax:

```
const removed = array.splice(start, deleteCount, item1, item2, ...);
```

Parameters:

Parameter	Description
start	Index where changes begin. Negative counts from end.
deleteCount	How many items to remove (use 0 to only insert)
item1...	(Optional) New items to insert at start position

Example:

```
const colors = ['red', 'green', 'blue', 'yellow'];

// Remove 1 item at index 1
const removed = colors.splice(1, 1);
console.log(removed); // what was taken out
console.log(colors); // modified array

// Insert 'purple' at index 1 (remove 0 items)
colors.splice(1, 0, 'purple');
console.log(colors);

// Replace index 2 with 'orange'
colors.splice(2, 1, 'orange');
console.log(colors);
```

```
■ ['green']
■ ['red', 'blue', 'yellow']
■ ['red', 'purple', 'blue', 'yellow']
■ ['red', 'purple', 'orange', 'yellow']
```

■ Remember: slice() = safe copy (non-destructive). splice() = surgery on the original (destructive). Easy to mix up!

Quick Cheat Sheet

Method	Mutates?	Use when you want to...
map()	No	Convert every item into something else
find()	No	Get the first item that matches a condition

findIndex()	No	Get the position (index) of the first match
filter()	No	Keep only items that pass a test
reduce()	No	Combine all items into one value (sum, object, etc.)
concat()	No	Join two or more arrays together
slice()	No	Copy a portion of an array safely
splice()	YES	Add, remove, or replace items inside an array

All non-mutating methods return a new array — the original is always safe to use again.